

COMPTE RENDU

Programmation Système - Projet N°1 - Processus et Ordonnancement
3^e année Cybersécurité — École Supérieure d'Informatique et du
Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha
Filière : Cybersécurité
Année : 2025/2026
Enseignants : M. Noufel & M. ElAmrani
Date : 22 février 2026

Remerciements

Ce mini projet a été réalisé dans le cadre du cours de Programmation Système dispensé par **M. Noufel** et **M. ElAmrani** à l'UIR durant l'année universitaire 2025-2026.

Je tiens à remercier les enseignants pour la qualité de leur enseignement, et notamment pour les travaux pratiques (TP0 à TP3) qui ont constitué les bases nécessaires à ce projet : gestion des processus, ordonnancement, signaux, et environnement shell Linux.

Je souhaite également remercier les enseignants des formations antérieures :

- Les professeurs de l'**Université Grenoble Alpes** (INF301, INF304, INF401) pour la rigueur en programmation et algorithmique ;
- Les professeurs de **Polytech Grenoble** (PROG C, CPS, ALG, ALM) pour la maîtrise du langage C et de la programmation système ;

Mohammed Taha HATHOUTI
Février 2026

Table des matières

1	Introduction	3
1.1	Contexte	3
1.2	Rappel de la consigne	3
1.3	Données du scénario	3
1.4	Rappels théoriques	3
2	Cas 1 : tous les processus arrivent à $t = 0$	4
2.1	FIFO (First In, First Out)	4
2.2	SJF (Shortest Job First) — sans préemption	4
2.3	SRTF (Shortest Remaining Time First) — avec préemption	4
2.4	Round Robin — quantum = 2	5
2.5	Comparaison — Cas 1	5
3	Cas 2 : arrivées échelonnées ($t = 0, 2, 4, 5$)	6
3.1	FIFO (First In, First Out)	6
3.2	SJF (Shortest Job First) — sans préemption	6
3.3	SRTF (Shortest Remaining Time First) — avec préemption	7
3.4	Round Robin — quantum = 2	7
3.5	Comparaison — Cas 2	8
4	Implémentation	8
4.1	Vue d'ensemble	8
4.2	Architecture du programme C	8
5	Analyse comparative	9
5.1	Synthèse globale	9
5.2	Analyse par algorithme	9
6	Conclusion	9

1 Introduction

1.1 Contexte

Dans un système d'exploitation multitâche, plusieurs processus coexistent en mémoire centrale et se partagent les ressources du processeur. L'**ordonnanceur** (scheduler) est le composant du noyau qui décide, à chaque instant, quel processus doit recevoir le CPU et pour combien de temps.

Le choix d'un algorithme d'ordonnancement a un impact direct et mesurable sur les performances globales du système : temps de réponse, équité entre processus, débit, et utilisation du processeur. L'étude et la comparaison de ces algorithmes est donc fondamentale pour comprendre le comportement d'un système d'exploitation moderne.

1.2 Rappel de la consigne

Le projet consiste à étudier le comportement de quatre algorithmes d'ordonnancement classiques — FIFO, SJF, SRTF et RR — appliqués à un scénario de quatre processus, et cela selon deux cas distincts :

- **Cas 1** : tous les processus arrivent à $t = 0$
- **Cas 2** : les processus arrivent à des instants échelonnés ($t = 0, 2, 4, 5$)

Les métriques calculées pour chaque algorithme sont le **temps de rotation moyen** (turnaround time) et le **temps moyen d'attente**, complétés de diagrammes de Gantt pour une visualisation claire de l'exécution.

1.3 Données du scénario

Processus	Temps CPU	Arrivée (Cas 1)	Arrivée (Cas 2)
P1	7	0	0
P2	4	0	2
P3	1	0	4
P4	4	0	5

La durée totale d'exécution est $7 + 4 + 1 + 4 = 16$ unités de temps.

1.4 Rappels théoriques

Temps de rotation (turnaround time) :

$$T_{\text{rotation}} = T_{\text{fin}} - T_{\text{arrivée}}$$

Temps d'attente :

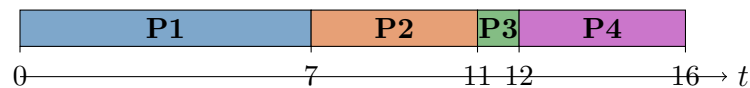
$$T_{\text{attente}} = T_{\text{rotation}} - T_{\text{CPU}}$$

2 Cas 1 : tous les processus arrivent à $t = 0$

Tous les processus sont disponibles dès $t = 0$, l'ordonnanceur a donc l'ensemble de la file dès le départ.

2.1 FIFO (First In, First Out)

FIFO exécute les processus dans leur ordre d'arrivée, sans préemption. Tous étant arrivés à $t = 0$, l'ordre d'exécution est P1, P2, P3, P4.



Processus	Fin	Rotation	Attente
P1	7	$7 - 0 = 7$	$7 - 7 = 0$
P2	11	$11 - 0 = 11$	$11 - 4 = 7$
P3	12	$12 - 0 = 12$	$12 - 1 = 11$
P4	16	$16 - 0 = 16$	$16 - 4 = 12$
Moyenne		$46/4 = 11.50$	$30/4 = 7.50$

FIFO souffre de l'**effet convoi** : P1 (burst = 7) bloque tous les autres, y compris P3 (burst = 1) qui attend 11 unités alors qu'il n'a besoin que d'1 unité de CPU.

2.2 SJF (Shortest Job First) — sans préemption

SJF choisit à chaque libération du CPU le processus avec le plus court burst disponible. Tous étant là à $t = 0$, l'ordre est déterminé par tri croissant des bursts : P3 (1), P2 (4), P4 (4), P1 (7). À égalité entre P2 et P4, l'ordre d'arrivée est conservé.



Processus	Fin	Rotation	Attente
P3	1	1	0
P2	5	5	1
P4	9	9	5
P1	16	16	9
Moyenne		$31/4 = 7.75$	$15/4 = 3.75$

2.3 SRTF (Shortest Remaining Time First) — avec préemption

SRTF est la version préemptive de SJF. À chaque arrivée d'un nouveau processus, si son temps restant est inférieur au processus courant, il prend le CPU. Comme tous les processus arrivent à $t = 0$, **aucune préemption ne peut se produire** : SRTF est identique à SJF dans ce cas.

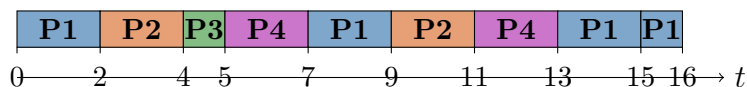
Processus	Fin	Rotation	Attente
P3	1	1	0
P2	5	5	1
P4	9	9	5
P1	16	16	9
Moyenne		$31/4 = 7.75$	$15/4 = 3.75$

2.4 Round Robin — quantum = 2

Round Robin attribue à chaque processus un quantum de 2 unités avant de passer au suivant. La file initiale est [P1, P2, P3, P4].

Déroulement :

- $t = 0 \rightarrow 2$: P1 exécute (reste 5), file = [P2, P3, P4, P1]
- $t = 2 \rightarrow 4$: P2 exécute (reste 2), file = [P3, P4, P1, P2]
- $t = 4 \rightarrow 5$: P3 exécute (reste 0) → **P3 finit à $t = 5$**
- $t = 5 \rightarrow 7$: P4 exécute (reste 2), file = [P1, P2, P4]
- $t = 7 \rightarrow 9$: P1 exécute (reste 3), file = [P2, P4, P1]
- $t = 9 \rightarrow 11$: P2 exécute → **P2 finit à $t = 11$**
- $t = 11 \rightarrow 13$: P4 exécute → **P4 finit à $t = 13$**
- $t = 13 \rightarrow 15$: P1 exécute (reste 1)
- $t = 15 \rightarrow 16$: P1 exécute → **P1 finit à $t = 16$**



Processus	Fin	Rotation	Attente
P1	16	16	9
P2	11	11	7
P3	5	5	4
P4	13	13	9
Moyenne		$45/4 = 11.25$	$29/4 = 7.25$

2.5 Comparaison — Cas 1

Algorithme	Moy. Rotation	Moy. Attente
FIFO	11.50	7.50
SJF	7.75	3.75
SRTF	7.75	3.75
RR ($q = 2$)	11.25	7.25

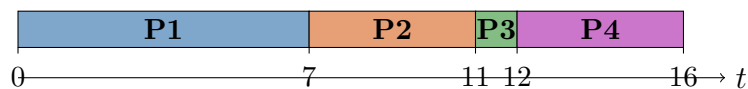
Quand tous les processus arrivent à $t = 0$, SJF et SRTF donnent les meilleurs résultats. FIFO est le moins performant à cause de l'effet convoi de P1.

3 Cas 2 : arrivées échelonnées ($t = 0, 2, 4, 5$)

Les processus n'arrivent pas tous en même temps. L'ordonnanceur doit gérer les arrivées dynamiquement et décider à chaque instant quel processus exécuter parmi ceux disponibles.

3.1 FIFO (First In, First Out)

L'ordre d'exécution est strictement l'ordre d'arrivée : P1, P2, P3, P4. P1 étant lancé dès $t = 0$, les autres attendent leur tour.



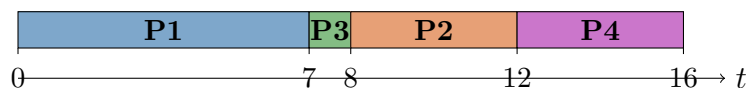
Processus	Fin	Rotation	Attente
P1	7	$7 - 0 = 7$	0
P2	11	$11 - 2 = 9$	5
P3	12	$12 - 4 = 8$	7
P4	16	$16 - 5 = 11$	7
Moyenne		$35/4 = 8.75$	$19/4 = 4.75$

P3 (burst = 1) attend 7 unités alors qu'il n'a besoin que d'1 unité de CPU : l'effet convoi est toujours présent.

3.2 SJF (Shortest Job First) — sans préemption

Déroulement :

- $t = 0$: seul P1 disponible $\rightarrow$ P1 s'exécute, finit à $t = 7$
- $t = 7$: disponibles = {P2 (burst=4), P3 (burst=1), P4 (burst=4)} $\rightarrow$ **P3** (plus court), finit à $t = 8$
- $t = 8$: disponibles = {P2 (burst=4), P4 (burst=4)} $\rightarrow$ **P2** (arrivé en premier), finit à $t = 12$
- $t = 12$: P4 $\rightarrow$ finit à $t = 16$



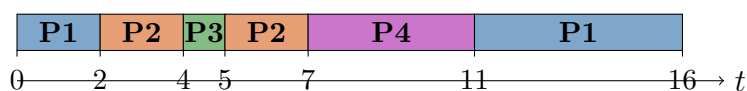
Processus	Fin	Rotation	Attente
P1	7	$7 - 0 = 7$	0
P3	8	$8 - 4 = 4$	3
P2	12	$12 - 2 = 10$	6
P4	16	$16 - 5 = 11$	7
Moyenne		$32/4 = 8.00$	$16/4 = 4.00$

3.3 SRTF (Shortest Remaining Time First) — avec préemption

SRTF vérifie à chaque arrivée si le nouveau processus a un temps restant inférieur au processus courant.

Déroulement détaillé :

- $t = 0$: seul P1 (restant=7) $\rightarrow$ P1 exécute
- $t = 2$: P2 arrive (restant=4), P1 restant=5 $>$ 4 $\rightarrow$ **préemption**, P2 prend le CPU
- $t = 4$: P3 arrive (restant=1), P2 restant=2 $>$ 1 $\rightarrow$ **préemption**, P3 prend le CPU
- $t = 5$: P3 finit à $t = 5$. P4 arrive (restant=4), P2 restant=2 $<$ 4 $\rightarrow$ P2 reprend
- $t = 7$: **P2 finit à $t = 7$** . Disponibles : P1 (restant=5), P4 (restant=4) $\rightarrow$ P4
- $t = 11$: **P4 finit à $t = 11$** . P1 (restant=5) $\rightarrow$ P1 reprend
- $t = 16$: **P1 finit à $t = 16$**



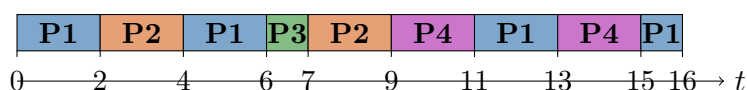
Processus	Fin	Rotation	Attente
P1	16	$16 - 0 = 16$	9
P2	7	$7 - 2 = 5$	1
P3	5	$5 - 4 = 1$	0
P4	11	$11 - 5 = 6$	2
Moyenne		$28/4 = 7.00$	$12/4 = 3.00$

SRTF donne le meilleur résultat global. P1 subit une attente de 9 unités au profit des processus plus courts : c'est le risque de **famine** inhérent à SRTF si des processus courts continuent d'arriver.

3.4 Round Robin — quantum = 2

Déroulement :

- $t = 0 \rightarrow 2$: P1 exécute (reste 5). À $t = 2$: P2 arrive $\rightarrow$ file = [P2, P1]
- $t = 2 \rightarrow 4$: P2 exécute (reste 2). À $t = 4$: P3 arrive $\rightarrow$ file = [P1, P3, P2]
- $t = 4 \rightarrow 6$: P1 exécute (reste 3). À $t = 5$: P4 arrive $\rightarrow$ file = [P3, P2, P4, P1]
- $t = 6 \rightarrow 7$: P3 exécute (reste 0) $\rightarrow$ **P3 finit à $t = 7$**
- $t = 7 \rightarrow 9$: P2 exécute (reste 0) $\rightarrow$ **P2 finit à $t = 9$**
- $t = 9 \rightarrow 11$: P4 exécute (reste 2), file = [P1, P4]
- $t = 11 \rightarrow 13$: P1 exécute (reste 1), file = [P4, P1]
- $t = 13 \rightarrow 15$: P4 exécute (reste 0) $\rightarrow$ **P4 finit à $t = 15$**
- $t = 15 \rightarrow 16$: P1 exécute (reste 0) $\rightarrow$ **P1 finit à $t = 16$**



Processus	Fin	Rotation	Attente
P1	16	$16 - 0 = 16$	9
P2	9	$9 - 2 = 7$	3
P3	7	$7 - 4 = 3$	2
P4	15	$15 - 5 = 10$	6
Moyenne		$36/4 = 9.00$	$20/4 = 5.00$

3.5 Comparaison — Cas 2

Algorithme	Moy. Rotation	Moy. Attente
FIFO	8.75	4.75
SJF	8.00	4.00
SRTF	7.00	3.00
RR ($q = 2$)	9.00	5.00

SRTF est clairement le plus efficace dans le Cas 2 grâce aux préemptions déclenchées par les arrivées successives de P2, P3 et P4.

4 Implémentation

4.1 Vue d'ensemble

Le projet est organisé en architecture modulaire. Chaque algorithme est isolé dans son propre fichier source, compilé séparément et lié via un `Makefile`.

4.2 Architecture du programme C

Listing 1 – Structure principale

```

1 typedef struct {
2     char nom[4];
3     int arrivee;
4     int burst;
5     int restant; /* pour SRTF et RR */
6     int fin;
7 } Processus;
```

Modules :

- `fifo.c` : exécution séquentielle par ordre d'arrivée
- `sjf.c` : sélection du processus de plus court burst disponible, sans préemption
- `srtf.c` : simulation tick par tick avec préemption sur temps restant minimal
- `rr.c` : file circulaire avec quantum configurable
- `affichage.c` : calcul et affichage des métriques
- `processus.c` : utilitaires (copie de tableau, réinitialisation)

Le programme accepte un fichier de jeu de test en argument. Format : une ligne par processus (`nom arrivee burst`).

Listing 2 – Exemples d'utilisation

```

1 make
2 ./projet1                               # cas 1 et cas 2 du projet
3 ./projet1 ../tests/effet_convoyi.txt    # jeu de test personnalis  

```

5 Analyse comparative

5.1 Synth  se globale

Algorithme	Cas 1 (t=0)		Cas 2 (��chelonn��)	
	Rot.	Att.	Rot.	Att.
FIFO	11.50	7.50	8.75	4.75
SJF	7.75	3.75	8.00	4.00
SRTF	7.75	3.75	7.00	3.00
RR ($q = 2$)	11.25	7.25	9.00	5.00

5.2 Analyse par algorithme

FIFO est le plus simple mais le moins performant. L'ordre d'arriv  e ne tient pas compte de la dur  e des processus, ce qui g  n  re l'**effet convoi**. Son avantage est l'absence de famine.

SJF am  liore les temps moyens en triant par dur  e croissante. Il est optimal pour minimiser le temps d'attente moyen quand tous les processus sont connus    l'avance. Les processus longs risquent la **famine** si des processus courts arrivent continuellement.

SRTF donne les meilleurs r  sultats, surtout dans le Cas 2 o   les arriv  es   chelonn  es permettent des pr  emptions b  n  fiques. Le risque de famine pour les processus longs est amplifi   (P1 attend 9 unit  s dans le Cas 2) et chaque pr  emption engendre une commutation de contexte.

Round Robin avec $q = 2$ offre la meilleure   quit   : chaque processus re  oit r  guli  rement du CPU et il n'y a pas de famine. Il ne minimise pas les temps moyens. Le quantum est un param  tre critique : trop petit il g  n  re trop de commutations, trop grand il d  g  n  re en FIFO.

6 Conclusion

Ce projet a permis d'analyser quatre algorithmes d'ordonnancement sur deux sc  narios et de les impl  menter en C en architecture modulaire. Les r  sultats confirment les propri  t  s th  oriques : SRTF minimise le temps d'attente moyen mais expose les processus longs    la famine ; FIFO est simple mais g  n  re l'effet convoi ; SJF est un bon compromis sans pr  emption ; Round Robin garantit l'  quit  . Le choix de l'algorithme d  pend du contexte : minimiser le temps de r  ponse, garantir l'  quit  , ou privil  gier la simplicit  .

Références

- **Cours** : M. Noufel & M. ElAmrani, *Programmation Système — Mini Projet N°1*, UIR ESIN, 2025-2026
- **Cours** : Prof. Eric GASCARD & Prof. Nicolas PALIX, *C Programmation Système (CPS)*, S5, Polytech Grenoble INP, 2024-2025
- **Cours** : Prof. Eric GASCARD, *Algorithmique Avancée*, S6, Polytech Grenoble INP, 2024-2025
- **Cours** : Prof. Michael PERIN, *Programmation C (PROG C)*, S5, Polytech Grenoble INP, 2024-2025
- **Cours** : Prof. Vincent DENJEAN, *Algorithmique (ALG)*, S5, Polytech Grenoble INP, 2024-2025
- **Cours** : Prof. Gwenaël DELAVAL, Responsable de l'UE INF304, *Bases du développement logiciel : modularisation, tests - C, L2 - S3*, Université Grenoble Alpes (UGA) 2023-2024
- **Cours** : Prof. Denis BOUHINEAU, *Architecture Matérielle (ALM)*, S4, Polytech Grenoble INP, 2023-2024
- **Cours** : Prof. Pascal SICARD & Prof. Akram IDANI, *Systèmes Embarqués*, S5, Polytech Grenoble INP, 2024-2025
- **Cours** : Prof. Pascal SICARD et al., *ALM S6 (Lustre, langages synchrones)*, S6, Polytech Grenoble INP, 2024-2025
- **TP's** : M. Noufel & M. ElAmrani, *TP0 à TP3 — Processus, Ordonnancement, Signaux et Shell Linux*, UIR ESIN, 2025-2026
- **Simulateur** : CPU Scheduling Simulator, <https://cpu-scheduling-sim.netlify.app/>
- **Ouvrage** : Abraham Silberschatz, Peter B. Galvin, Greg Gagne, *Operating System Concepts*, 10^e édition, Wiley, 2018
- **Assistant IA** : Claude (Sonnet 4.6), Anthropic, <https://claude.ai>